

PROJECT DOCUMENTATION

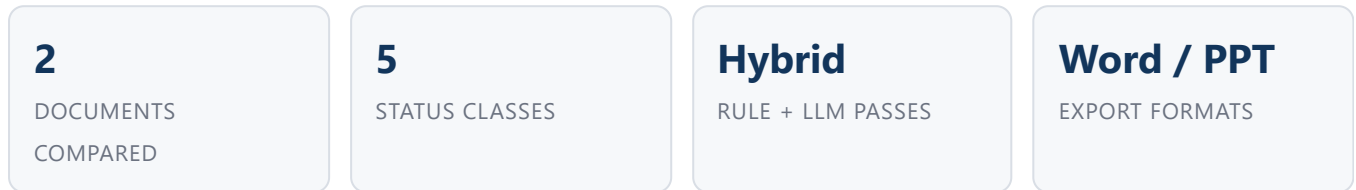
Status Report App

An AI-assisted platform that compares a project **proposal** against a delivered **status report**, classifies every planned item, and produces presentation-grade, module-wise Word & PowerPoint reports.

Technical & functional documentation · Generated 29 June 2026 · Spring Boot 3.3 / Java 21 · React + TypeScript

1. Overview

The Status Report App answers a single recurring question for project managers: *"We promised X in the proposal — what actually got done according to the latest status report?"* It ingests both documents, extracts discrete work items from each, semantically matches them, assigns a delivery status to every promised item, and exports a polished report.

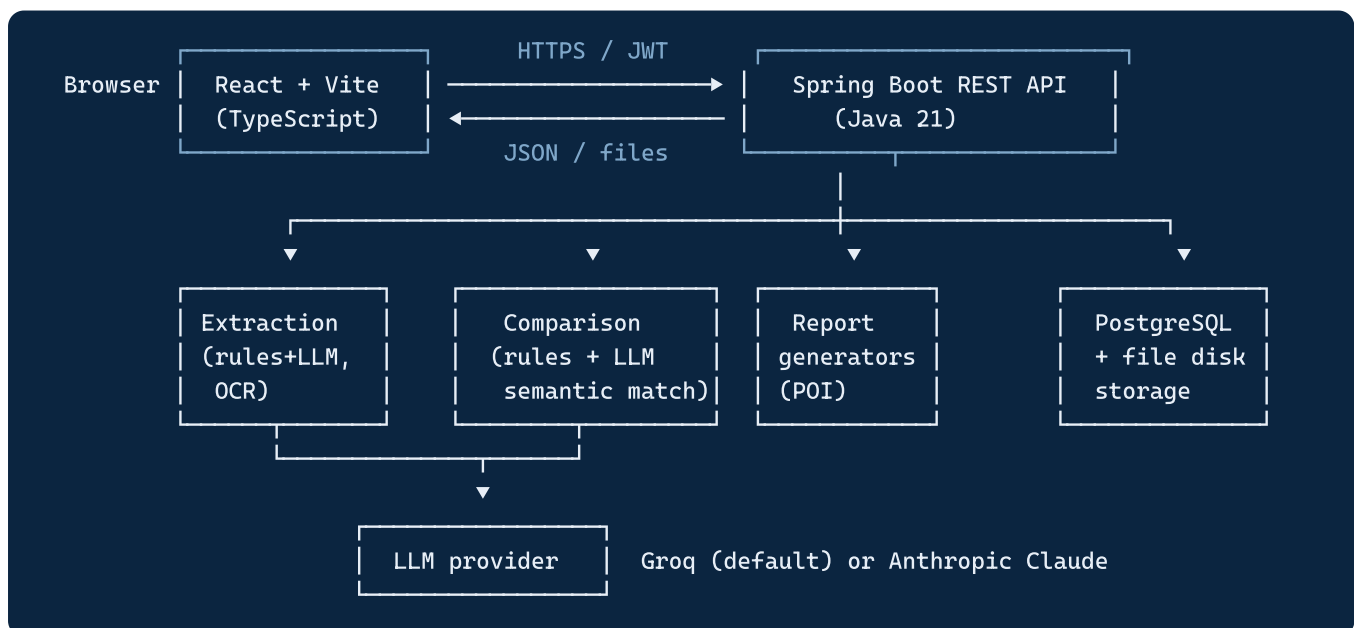


What it does, end to end

- **Upload** a proposal and one or more status reports (PDF or DOCX).
- **Extract** work items (tasks, milestones, deliverables, risks) from each document, including a **module** (functional area) inferred for each item.
- **Compare** proposal items against status-report items and classify each.
- **Report** the verdicts as a branded, **module-wise** Word document or PowerPoint deck, stamped with the **reporting period** derived from item dates.

2. Architecture

A classic two-tier web application: a React single-page frontend talking to a stateless Spring Boot REST API, backed by PostgreSQL, with an external LLM provider used only as a fallback for the cases deterministic rules can't resolve.



Tech stack

Layer	Technology	Notes
Backend	Spring Boot 3.3, Java 21	REST controllers, JPA, Spring Security
Frontend	React + TypeScript, Vite	SPA with token auth, file dropzones
Database	PostgreSQL	JPA / Hibernate, <code>ddl-auto: update</code>
Auth	JWT (jjwt)	Stateless bearer tokens, BCrypt passwords
Documents	Apache POI, PDFBox	DOCX/PDF read & Word/PPT write
OCR	Tess4J (Tesseract)	Fallback for scanned, image-only PDFs
LLM	Groq (default) / Anthropic Claude	One client bean loads per <code>llm.provider</code>

3. Extraction pipeline

Each uploaded document is converted to text, then mined for work items by a two-pass hybrid orchestrator. The goal: handle templated documents for free with cheap rules, and only spend LLM tokens on free-form narrative.

Stage	Component	Responsibility
Text extraction	<code>TextExtractorFactory</code> → <code>DocxTextExtractor</code> / <code>PdfTextExtractor</code>	Pull raw text from DOCX/PDF
OCR fallback	<code>PdfOcrService</code>	Image-only PDFs with no text layer (Tesseract)
Pass 1 — rules	<code>RuleBasedExtractor</code>	Bullet / numbered lists, inline dates; cheap & deterministic
Pass 2 — LLM	<code>LlmExtractor</code>	Free-form prose the rules left over; infers module per item
Orchestration	<code>ExtractionOrchestrator</code>	Runs rules first, sends only leftovers to the LLM, merges & persists

Module inference. The LLM extraction prompt asks the model to tag each item with a **module** — its functional area or workstream (e.g. *Authentication, Reporting, Infrastructure*). Items the rule pass produces, or that the model can't classify, fall under **"General"**. Modules drive the grouping in the generated reports.

Robustness details: large narratives are chunked before being sent to the model so a single reply never overruns the token budget; if a reply is truncated mid-array, `LlmExtractor` salvages every complete JSON object it can rather than discarding the batch.

4. Comparison & status classification

Once both documents are extracted, `ComparisonOrchestrator` matches proposal items to status-report items using the same hybrid shape: a deterministic `RuleBasedMatcher` first, then an `LlmSemanticMatcher` for the leftovers. Re-running a comparison **replaces** the previous verdicts rather than appending.

Every proposal item ends up with one of five statuses (an item never mentioned in the status report is recorded as `NOT STARTED` with a null match):

Status	Meaning
<code>COMPLETED</code>	Delivered per the status report
<code>IN PROGRESS</code>	Underway, not yet finished
<code>PENDING</code>	Acknowledged but not started
<code>AT RISK</code>	Flagged as blocked, delayed or in jeopardy
<code>NOT STARTED</code>	Promised item the status report never mentions

5. Reporting

Two presentation-grade exports are produced from the saved comparison results, both written with Apache POI and both organised **by module**.

Word report — `WordReportGenerator`

- Branded cover with the project name, description, **reporting period** and generation date.
- “At a glance” KPI strip and an overall status-distribution breakdown (project-wide).
- “**Item details by module**” — one sub-section per module, each with its own item count, % complete, and a colour-coded details table (planned item · status · evidence).

PowerPoint deck — `PptReportGenerator`

- 16:9 cover and a summary slide (KPI tiles + stacked distribution bar).
- Detail slides grouped **by module** (≤ 6 items per slide), each row carrying a colour-coded status label and the supporting evidence.
- Cover and footers stamped with the reporting period and page numbers.

Reporting period (auto-derived). Rather than asking the user for dates, the report computes a window from the items themselves: **start = earliest** and **end = latest** planned date across the tracked items. When no item carries a planned date, the cover simply shows the generation date. Within a report, named modules are listed alphabetically and the “General” catch-all is always rendered last.

6. Data model

Entity	Key fields	Purpose
User	username, passwordHash	Authentication principal
Project	name, description, createdAt	Workspace tying documents & results together
ProposalDocument / StatusReportDocument	originalFilename, storagePath, rawText	Uploaded files + extracted text
ProjectItem	title, description, plannedDate, module , source, extractionMethod	One extracted work item
ComparisonResult	proposalItem, statusItem, status, evidence, matchedBy	One classified verdict

The `module` column was added to `ProjectItem`; Hibernate's `ddl-auto: update` creates it automatically on startup. Items extracted before this change have a null module and are reported under "General".

7. REST API reference

All endpoints are under `/api` and (except auth) require a `Authorization: Bearer <token>` header.

Method	Path	Purpose
POST	<code>/api/auth/register</code>	Create an account, returns a JWT
POST	<code>/api/auth/login</code>	Authenticate, returns a JWT
GET	<code>/api/auth/me</code>	Current user from the token
GET / POST	<code>/api/projects</code>	List / create projects
GET / DELETE	<code>/api/projects/{id}</code>	Fetch / delete a project
POST	<code>/api/projects/{id}/proposal</code>	Upload the proposal (multipart <code>file</code>)
POST	<code>/api/projects/{id}/status-reports</code>	Upload a status report (multipart <code>file</code>)
GET	<code>/api/projects/{id}/items</code>	See what was extracted from each document
POST	<code>/api/projects/{id}/comparison/run</code>	Run the comparison (replaces previous results)
GET	<code>/api/projects/{id}/comparison</code>	Get the last comparison results
GET	<code>/api/projects/{id}/report/word</code>	Download the Word report (.docx)
GET	<code>/api/projects/{id}/report/ppt</code>	Download the PowerPoint deck (.pptx)

8. Configuration

All settings are environment-overridable (see `application.yml`). Key variables:

Variable	Default	Purpose
DB_URL / DB_USERNAME / DB_PASSWORD	local Postgres	Database connection
LLM_PROVIDER	groq	groq or anthropic
GROQ_API_KEY / GROQ_MODEL	— / llama-3.1-8b-instant	Groq provider credentials & model
ANTHROPIC_API_KEY	—	Claude provider credentials (when selected)
OCR_ENABLED / TESSDATA_PATH / OCR_LANGUAGE / OCR_DPI	true / tessdata / eng / 300	Tesseract OCR fallback
JWT_SECRET / JWT_EXPIRATION_MS	dev secret / 24h	Token signing (use ≥ 32-char secret in prod)
APP_STORAGE_DIR	./uploaded-documents	Where uploaded files are stored
APP_CORS_ALLOWED_ORIGINS	localhost:5173	Allowed frontend origins

Running locally

```
# 1) Start PostgreSQL (Docker compose)
docker compose up -d

# 2) Provide an LLM key (Groq by default)
export GROQ_API_KEY=gsk_...

# 3) Run the backend (Java 21) – API on http://localhost:8080
mvn spring-boot:run

# 4) Run the frontend dev server – http://localhost:5173
cd frontend && npm install && npm run dev
```

9. Recent changes

- **Module-wise reports.** A `module` field was added to items (inferred by the LLM extractor); both the Word and PowerPoint generators now group item detail by module.
- **Reporting period.** Report covers now show a start–end window auto-derived from the earliest and latest item planned dates.
- **PostgreSQL @Lob fix.** Removed `@Lob` from the `TEXT` string columns (`evidence` , `description` , `rawText`). On Postgres, `@Lob` routes reads through the large-object API, which fails in auto-commit mode (*"Large Objects may not be used in auto-commit mode"*). The columns were already `TEXT` , so no migration was needed — only the read binding changed.

